Document number:	P0357R2
Date:	2018-06-10
Project:	Programming Language C++, Library Working Group
Reply-to:	*Tomasz Kamiński <tomaszkam at gmail dot com>*
	Stephan T. Lavavej <stl at microsoft dot com>
	Alisdair Meredith <ameredith1 at bloomberg dot net>

`reference_wrapper` for incomplete types

1. Introduction

This paper proposes a change in `reference_wrapper`'s specification to support incomplete types.

2. History

2.1. Revision 2

- Updated wording to N4750 (C++ Working Draft, 2018-05-07).
- Dropped uncessary removal wording (accepted as part of P0619R4).
- Updated discussion section of paper to match current state of C++20.

2.1. Revision 1

- Minor corrections and clarifications to paper text. Clarified relation to P0091: Template argument deduction for class templates
- Updated wording to refer to N4606 (C++ Working Draft, 2016-07-12) and included content of removed subclause.
- Included proposed feature testing recommendation macro.
- Corrected included implementation for the `operator()` of `reference_wrapper`, so it is now const qualified.
- Extended implementability section to discuss details of instantiation of `reference_wrapper` and `operator()` member.

3. Motivation and Scope

`std::reference_wrapper` is a utility class created to allow references to be used in interfaces that were designed to pass objects by value. However the design of the standard component has a major drawback, when compared to the alternative solutions based on the use of the raw pointers of boost version of this component: the referenced type is required to be complete. As a consequence, depending on the context use of `reference_wrapper` may increase compilation time by adding a new definition, or even be impossible in the case when the definition of the class is not available to the programmer.

Moreover `std::reference_wrapper` specializations are recognized by standard factory functions, like: `std::make_pair`, `std::make_tuple` or `std::bind` which allow the programmer to create pairs of references by use of:

```
auto p = std::make_pair(std::ref(t), std::ref(u));
```

Use of this feature, not only avoids cumbersome specification of the type, but also eliminates the possibility of encountering dangling reference problems that it may introduce. For example in the case of the following definition:

```
std::pair<std::string const&, int> p("test", 10);
```

every use of `p.first` leads to undefined behaviour caused by reading a dangling reference. Despite all of the above advantages, programmers are still forced to use `pair<T&, U&>`, when at least one of the types `T` or `U` is incomplete.

Furthermore this problem is not addressed by inclusion of the P0091: Template argument deduction for class templates, as implicit deduction guides synthesized from `pair` and `tuple` constructors do not deduce reference types.

4. Design Decisions

Supporting incomplete types in `reference_wrapper` was impossible until recent removal of `result_type` and related typedefs introduced by P0619: Reviewing Deprecated Facilities of C++17 for C++20, as to provide them, implementations were required to check the template parameter for their presence.

4.1. Forced removal of nested typedefs

The feature proposed in this paper conflicts with support for the old function binding protocol and vendors will no longer be allowed to provide required typedefs in their `std::reference_wrapper` implementations.

4.2. Support for `is_transparent`

In C++14, another protocol based on the presence of the `is_transparent` nested type was introduced, to indicate that a given functor enables heterogeneous lookup for associative container. As in the case of `result_type` implementation of this protocol in exact form for `reference_wrapper<T>` would reintroduce requirement of completeness of `T` template parameter.

Despite the fact that support for incomplete types and heterogeneous container lookup in `reference_wrapper` may look incompatible, there is the possibility to provide both of them, via an alternative design that relies on a metafunction instead of a nested type, as proposed in P0046R1: Change is_transparent to metafunction (Revision 1).

5. Impact On The Standard

This proposal depends on the removal of the `result_type` and related typedefs for C++20.

Nothing depends on this proposal.

6. Proposed Wording

The proposed wording changes refer to N4750 (C++ Working Draft, 2018-05-07).

At the end of the section 23.14.5 Class template `reference_wrapper` [refwrap]:

> The template parameter `T` of `reference_wrapper` may be an incomplete type.

Apply following changes to paragraph 23.14.5.4 `reference_wrapper` invocation [refwrap.invoke]:

> ```
> template <class... ArgTypes>
>   result_of_t<T&(ArgTypes&&... )>
>     operator()(ArgTypes&&... args) const;
> ```
>
> *Returns:* *INVOKE*`(get(), std::forward<ArgTypes>(args)...)`. ([func.require] 20.12.2).
>
> *Remarks:* If `T` is an incomplete type, the program is ill-formed.

At the beginning of the paragraph 23.14.4.5 `reference_wrapper` helper functions [refwrap.helpers]:

> The template parameter `T` of the following `ref` and `cref` function templates may be an incomplete type.

7. Feature-testing recommendation

For the purposes of SG10, we recommend the macro name `__cpp_lib_reference_wrapper` with value `20`*YYMM* representing publication date, to be defined in the `<functional>` header. The intent is to allow reuse of the same macro to indicate presence of the original feature from C++11 standard.

Usage example:

```
template<typename T, typename U>
auto my_tie(T& t, U & u)
{
#if __cpp_lib_reference_wrapper >= 20YYMM
  return std::make_pair(std::ref(t), std::ref(u));
#else
  return std::pair<T&, U&>(t, u);
#endif
}
```

8. Implementability

Without requirement to conditionally support `result_type` and related typedefs, straightforward implementation provides support for incomplete types.

```
template<typename T>
class reference_wrapper
{
  T* ptr;

public:
  using type = T;

  reference_wrapper(T& val) noexcept
     : ptr(std::addressof(val))
  {}

  reference_wrapper(T&&) = delete;

  T& get() const noexcept { return *ptr; }
  operator T&() const noexcept{ return *ptr; }

  template<typename... Args>
  auto operator()(Args&&... args) const
    -> std::result_of_t<T&(Args...)>
  { return std::invoke(*ptr, std::forward<Args>(args)...); }
};
```

Careful reader may notice, that the `operator()` requires template parameter `T` to be a complete type, and this requirement is not only limited to definition of the function, but also its declaration, that uses `std::result_of_t<T&(Args...)>` to specify return type. However, call operator is an template function member of the class and its declaration will not be instantiated during the instantiation of enclosing `reference_wrapper` specialization, as the `Args` template parameter pack are not know at this point. As a consequence the user is allowed to use an object of `reference_wrapper<T>` with `T` being an incomplete type, unless it is actually called.

9. Acknowledgements

Special thanks and recognition goes to Sabre (http://www.sabre.com) for supporting the production of this proposal, and for sponsoring Tomasz Kamiński's trip to the Oulu for WG21 meeting.

10. References

1. Tomasz Kamiński, Stephan T. Lavavej, Alisdair Meredith, "Reviewing Deprecated Facilities of C++17 for C++20", (P0619R4, https://wg21.link/p0619r4)
2. Tomasz Kamiński, "Change is_transparent to metafunction (Revision 1)", (P0046R1, http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0046r1.html)
3. Mike Spertus, Faisal Vali, Richard Smith, "Template argument deduction for class templates (Rev. 6)", (P0091R3, http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0091r3.html)
4. Richard Smith, "Working Draft, Standard for Programming Language C++" (N4750, http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4750.pdf)